

实验五

一、实验环境

Ubuntu 22.04.05

GCC 11.4.0

Flex 2.6.4

Bison 3.8.2

二、完成功能

本实验在实验三生成的中间代码（IR）基础上，实现了对 IR 文本的读取、解析及若干局部中间代码优化。当前已完成的功能如下：

1. IR 文本解析与重建

实现了从中间代码文本文件（`.ir`）中逐行读取指令，并将其解析为程序内部统一使用的中间代码表示（`IRNode` + `IRList`）。

支持的指令类型包括：

- 函数与标签指令：`FUNCTION`、`LABEL`
- 算术与赋值指令：`:=`、`+`、`-`、`*`、`/`
- 地址与内存访问指令：`&`、`*`
- 控制流指令：`GOTO`、`IF`
- 函数调用与参数传递：`CALL`、`ARG`、`PARAM`
- 输入输出指令：`READ`、`WRITE`
- 返回指令：`RETURN`

解析后的 IR 能够重新输出为合法的中间代码文本，为后续优化和代码生成提供统一的数据结构基础。

2. 局部常量传播与常量折叠优化

在不构建控制流图的前提下，实现了基于顺序扫描的局部常量传播与常量折叠优化：

- 维护变量/临时变量到常量值的映射表；
- 对形如 `x := #c` 的指令记录常量信息；
- 对形如 `x := y op z` 的指令，在操作数均为常量时进行编译期计算（常量折叠），将其简化为 `x := #k`；
- 在变量被重新赋值、读入（`READ`）、内存写入（`STORE`）或函数调用（`CALL`）等可能引入不确定性的场景下，保守地清空或更新常量信息，保证程序语义正确。

该优化有效减少了运行时不必要的算术计算，提高了中间代码的执行效率。

3. 局部无用代码消除（Dead Code Elimination）

在常量传播之后，实现了一趟**局部无用代码消除**（Local DCE）：

- 采用从后向前扫描的方式维护“已使用变量集合”；
- 对不产生副作用、且其定义结果在后续未被使用的赋值类指令进行删除；
- 保留所有具有副作用或控制流影响的指令（如 `READ`、`WRITE`、`CALL`、`IF`、`GOTO`、`RETURN` 等）；
- 正确处理变量的重新定义，避免误删被后续定义覆盖的语句。

该优化显著减少了冗余的临时变量赋值与无效计算，使生成的中间代码更加紧凑。